LLM / LAB NOTE 01

DATA → TOKENIZER → GPT → TRAIN → EVAL → API

大模型预训练与部署实战教程

用一个约 14M 参数的开源 GPT 式模型，完整跑通从原始文本到 HTTP 推理服务的工程闭环

目标
从零初始化权重，不下载预训练模型

数据
TinyStories + 自训练 Byte-level BPE

训练
PyTorch / Transformers / 单卡优先

部署
FastAPI 最小服务 + vLLM 并发服务

先跑通一个可复现闭环

这条路线故意不从 7B 模型开始。先在一张消费级 GPU，甚至 CPU 冒烟环境中，把“数据可追溯、分词器可复用、训练可恢复、评估可比较、服务可调用”五件事一次做对，再扩规模。

最终会得到

- `artifacts/tokenizer/`：自训练词表与特殊词元配置
- `artifacts/checkpoints/`：可断点续训的中间检查点
- `artifacts/final/`：Transformers / safetensors 格式模型
- `/generate`：最小 HTTP 文本生成接口
- `/v1/completions`：vLLM 的 OpenAI 兼容接口

明确不承诺

- 这不是聊天助手：只做基础因果语言建模，没有指令微调。
- 小模型不会获得通用世界知识；它主要学习 TinyStories 的英文故事分布。
- 教程中的显存与耗时只能当预算起点，最终以你机器实测为准。
- 上线前仍需完成数据许可、安全、压测与访问控制审查。

最短成功路径

1. **20k 样本冒烟：**验证环境、数据字段、loss 下降与保存/加载。
2. **200k 样本基线：**训练 5k steps，固定验证集与生成提示。
3. **API 单实例：**FastAPI 验证模型能被业务代码调用。
4. **GPU 并发：**需要吞吐时切 vLLM，再做压测和限流。

验收底线：能从 checkpoint 恢复；同一提示在固定 seed 下可复现；验证 loss 没有持续恶化；服务重启后健康检查与生成接口都能返回。

把问题拆成八个可验证层



预训练到底在优化什么

给定词元序列 $x_1 \cdots x_t$，模型用因果掩码只看当前位置及其左侧，并最大化每个下一词元的条件概率：

$$L = -\Sigma_t \log p(x_t \mid x_{<t}) \quad PPL = \exp(\text{平均 token loss})$$

模型内部

- Embedding 把 token ID 映射为向量。
- 位置编码告诉模型顺序。
- Masked self-attention 计算上下文。
- MLP、残差与归一化稳定表示。
- LM head 输出整个词表的 logits。

工程外部

- Tokenizer 决定模型“看到”的基本单位。
- 数据分布决定模型学会什么。
- batch、精度和序列长决定显存与吞吐。
- 验证切分和提示集决定你能否发现过拟合。
- 服务层决定模型能否安全地被使用。

关键区别：预训练学“续写分布”；监督微调（SFT）学“按指令回答”；RAG 给模型提供外部检索上下文。不要用基础预训练结果直接宣称“聊天模型”。

先定实验预算，再碰超参数

三档实验预算

档位	模型与数据	建议资源	用途
冒烟	约 14M；20k stories；512 steps	CPU 可运行；GPU 4 GB 更顺畅	查依赖、字段、shape、保存/加载
基线	约 14M；200k stories；5k-30k steps	单卡 8-12 GB；磁盘预留 10 GB+	得到可读英文故事与稳定曲线
扩展	约 90M；完整或自有语料	24 GB 级单卡或多卡；更大磁盘	验证 DDP/FSDP、数据吞吐与服务并发

资源数字是预算起点，不是性能保证。显存还受序列长度、micro-batch、激活、优化器状态、PyTorch/CUDA 版本影响。

默认 14M 配置

- 词表：8,000
- 上下文：256 tokens
- 层数：6
- 隐藏维度：384
- 注意力头：6
- FFN：1,536

约 90M 扩展配置

- 词表：16,000 或 32,000
- 上下文：512
- 层数：12
- 隐藏维度：768
- 注意力头：12
- FFN：3,072

训练显存怎么估

仅参数权重在 BF16 下约为 `2 × 参数量` 字节；训练还要加梯度、Adam 一阶/二阶状态、可能的 FP32 master weights 和激活。实践中先把 micro-batch 设为 2 或 4，开启梯度累积与 gradient checkpointing，再逐步加大。

> **扩展顺序：**先增加数据 token 数，再观察验证曲线；随后增加训练 steps；最后才增模型。每次只改一个主变量，并保留上一版可回滚 checkpoint。

先让环境可证明地正确

系统与软件

- ✓ Linux 优先；Windows 建议 WSL2。Python 3.10-3.12。
- ✓ NVIDIA GPU：先让 `nvidia-smi` 正常，再按官方选择器安装与驱动匹配的 PyTorch。
- ✓ 至少为语料、缓存、tokenized dataset 和 checkpoints 各留一份空间。
- ✓ Git、curl、Docker（部署阶段可选）；不要在系统 Python 里直接 pip。

创建隔离环境

```
mkdir tinystory-gpt && cd tinystory-gpt
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip wheel

# 先按你的 CPU/CUDA 环境安装 torch，再安装其余依赖
pip install "transformers>=4.49,<6" "datasets>=3,<5" \
  "tokenizers>=0.20" "accelerate>=1" "safetensors>=0.4" \
  "tensorboard>=2.16" "fastapi>=0.115" "uvicorn[standard]>=0.30"
```

环境自检

```
python -c "import torch; print(torch.__version__); print(torch.cuda.is_available())"
python -c "import transformers,datasets,tokenizers; print(transformers.__version__, datasets.__version__,
tokenizers.__version__)"
pip freeze > requirements.lock.txt
```

不要复制别人的 CUDA wheel 命令。CUDA 运行时、PyTorch wheel 和 GPU 驱动不匹配，是最常见的“装得上但看不到 GPU”原因。vLLM 更应使用独立新环境，避免二进制依赖冲突。

目录结构就是实验边界

项目布局

```
tinystory-gpt/
├── data/                    # 原始/抽样 JSONL
├── artifacts/
│   ├── tokenizer/           # tokenizer.json + configs
│   ├── tokenized/           # datasets.save_to_disk
│   ├── checkpoints/         # checkpoint-NNNN
│   └── final/               # 最终推理目录
├── src/
│   ├── prepare_data.py
│   ├── train_tokenizer.py
│   ├── tokenize_data.py
│   ├── train.py
│   ├── evaluate.py
│   ├── generate.py
│   └── serve_api.py
├── requirements.txt
└── requirements.lock.txt
```

全流程命令顺序

```
python src/prepare_data.py --train 200000 --valid 5000
python src/train_tokenizer.py --vocab-size 8000
python src/tokenize_data.py --block-size 256
python src/train.py --max-steps 5000
python src/evaluate.py
python src/generate.py --prompt "Once upon a time"
uvicorn src.serve_api:app --host 0.0.0.0 --port 8000
```

版本化哪些东西

- 脚本与配置
- 数据来源、许可与快照 ID
- tokenizer 文件及哈希
- 固定验证集和提示集
- 依赖锁与运行命令

不要提交哪些东西

- 未获授权的原始语料
- 个人信息和密钥
- 超大 checkpoints
- 本机缓存目录
- 日志里的输入正文

可复现单位：不是“一个 .py 文件”，而是数据快照 + tokenizer + 模型 config + 训练参数 + 随机种子 + 软件版本 + checkpoint。

数据质量先于模型大小

本教程使用 TinyStories，因为它是英文短故事语料、词汇受限，适合验证小模型的完整预训练流程。数据集卡说明其文本由 GPT-3.5/4 合成，并给出 CDLA-Sharing-1.0 许可元数据；真正用于生产前仍需自行复核许可与用途。

数据进入训练前的五道门

门	检查	产物
权利	来源、许可、隐私、地域与用途限制	DATA_CARD.md
格式	UTF-8、字段 schema、空值、异常长文本	schema + 失败样本
质量	乱码、模板垃圾、语言、毒性、重复段落	过滤规则与计数
去重	文档级 hash；近重复可用 MinHash/LSH	去重前后数量
切分	先去重再 train/valid；避免同源泄漏	固定验证快照

`prepare_data.py`（上半）

```
import argparse, hashlib, json
from itertools import islice
from pathlib import Path
from datasets import load_dataset
DATASET = "roneneldan/TinyStories"
def clean(text: str) -> str:
    text = text.replace("\r\n", "\n").strip()
    return "\n".join(line.rstrip() for line in text.splitlines())
def dump(split: str, limit: int, out: Path):
    ds = load_dataset(DATASET, split=split, streaming=True)
    seen, kept = set(), 0
    out.parent.mkdir(parents=True, exist_ok=True)
    with out.open("w", encoding="utf-8") as f:
        for row in islice(ds, limit * 2):
            text = clean(row.get("text", ""))
            if len(text) < 40 or len(text) > 12000: continue
            digest = hashlib.sha256(text.encode()).hexdigest()
            if digest in seen: continue
            seen.add(digest)
            f.write(json.dumps({"text": text}, ensure_ascii=False) + "\n")
            kept += 1
            if kept >= limit: break
    print(split, "kept=", kept)
```

把语料变成可审计的快照

`prepare_data.py`（下半）

```
if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("--train", type=int, default=200_000)
    ap.add_argument("--valid", type=int, default=5_000)
    args = ap.parse_args()
    dump("train", args.train, Path("data/train.jsonl"))
    dump("validation", args.valid, Path("data/valid.jsonl"))
```

运行与抽检

```
python src/prepare_data.py --train 200000 --valid 5000
wc -l data/*.jsonl
python -c 'import json; print(json.loads(open("data/train.jsonl").readline())["text"][:300])'
sha256sum data/*.jsonl > data/SHA256SUMS
```

必须记录

- 数据集名称与 revision/快照
- 抓取或生成日期
- 过滤、去重与切分代码版本
- 原始数、保留数、拒绝数及原因
- 训练/验证文件 SHA-256

自有语料替换

- 最终统一成每行一个 `{"text": ...}`
- 按来源保留 metadata 供审计
- 个人信息先脱敏再训练
- 对话、代码、网页分别清洗
- 不要让同一文档跨 train/valid

流式数据的限制： `streaming=True` 返回可迭代数据集，不支持普通随机访问。教程先落地成 JSONL，便于复跑、校验和训练 tokenizer；超大规模时再改为真正流式管线。

分词器是模型协议，不是前处理细节

为什么自训练 Byte-level BPE

- **Byte-level：**从字节字母表出发，通常不因生僻字符直接产生未知词。
- **BPE：**反复合并高频相邻符号，在词表大小与序列长度间折中。
- **同域：**词表从训练语料学习，避免拿完全不同领域的 token 切分。
- **不可后改：**模型训练后改 token ID 会让 embedding 与词义错位。

`train_tokenizer.py`

```
import argparse, json
from pathlib import Path
from tokenizers import Tokenizer, decoders, models, normalizers, pre_tokenizers, trainers
from transformers import PreTrainedTokenizerFast
SPECIAL = ["<pad>", "<bos>", "<eos>", "<unk>"]
def texts(path):
    with open(path, encoding="utf-8") as f:
        for line in f: yield json.loads(line)["text"]
ap = argparse.ArgumentParser(); ap.add_argument("--vocab-size", type=int, default=8000)
args = ap.parse_args()
backend = Tokenizer(models.BPE(unk_token="<unk>"))
backend.normalizer = normalizers.NFKC()
backend.pre_tokenizer = pre_tokenizers.ByteLevel(add_prefix_space=False)
backend.decoder = decoders.ByteLevel()
trainer = trainers.BpeTrainer(vocab_size=args.vocab_size, min_frequency=2,
    special_tokens=SPECIAL,
    initial_alphabet=pre_tokenizers.ByteLevel.alphabet())
backend.train_from_iterator(texts("data/train.jsonl"), trainer=trainer)
```

保存后重新加载，才算分词器验收

`train_tokenizer.py`（保存与验收）

```
out = Path("artifacts/tokenizer"); out.mkdir(parents=True, exist_ok=True)
fast = PreTrainedTokenizerFast(tokenizer_object=backend,
    pad_token="<pad>", bos_token="<bos>", eos_token="<eos>",
    unk_token="<unk>", model_max_length=256)
fast.save_pretrained(out)
for text in ["Once upon a time, there was a small robot.",
             "The dragon couldn't open the blue door!"]:
    ids = fast.encode(text, add_special_tokens=False)
    print(len(ids), ids[:20], fast.decode(ids))
print("vocab=", len(fast), "special=", fast.special_tokens_map)
```

执行与质量检查

```
python src/train_tokenizer.py --vocab-size 8000
python - <<'CHECK'
from transformers import AutoTokenizer
t=AutoTokenizer.from_pretrained("artifacts/tokenizer")
assert len({t.pad_token_id,t.bos_token_id,t.eos_token_id,t.unk_token_id}) == 4
s="Once upon a time, Lily found a bright key."
ids=t.encode(s, add_special_tokens=False)
print(t.convert_ids_to_tokens(ids)); print(t.decode(ids))
CHECK
```

指标	怎么看	异常意味着
round-trip	decode(encode(text)) 基本保真	normalizer/decoder 配置错误
UNK 率	Byte-level 下应接近 0	alphabet 或 unknown 处理有问题
chars/token	同一固定样本横向比较	过低：词表太小；过高：可能过拟合
special IDs	唯一、固定、保存后不变	模型与 tokenizer 会错位

> **中文语料：**仍可使用 Byte-level BPE，但 8k 词表可能让序列过长。先统计 chars/token 与长度分布，再尝试 16k/32k；不要凭习惯直接把词表放大。

固定长度打包换来更高 token 利用率

为什么要拼接再切块

如果逐故事 pad 到 256，短样本会浪费大量计算。更高效的做法是：每个故事末尾加 `<eos>`，把 token 串拼接，然后切成固定长度 block。模型能在 EOS 处学习文档边界。

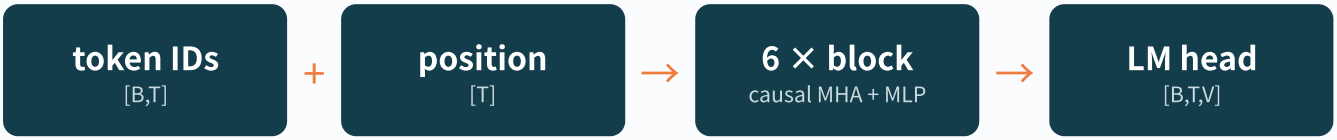
`tokenize_data.py`

```
import argparse
from datasets import load_dataset
from transformers import AutoTokenizer
ap=argparse.ArgumentParser(); ap.add_argument("--block-size", type=int, default=256)
ap.add_argument("--num-proc", type=int, default=4); a=ap.parse_args()
tok=AutoTokenizer.from_pretrained("artifacts/tokenizer")
assert a.block_size <= tok.model_max_length
raw=load_dataset("json", data_files={"train":"data/train.jsonl",
                                     "validation":"data/valid.jsonl"})
def encode(batch): return tok(batch["text"], add_special_tokens=False, truncation=False)
tokenized=raw.map(encode, batched=True,
    remove_columns=raw["train"].column_names, num_proc=a.num_proc)
def group(batch):
    stream=[]
    for ids in batch["input_ids"]:
        stream.extend(ids); stream.append(tok.eos_token_id)
    usable=(len(stream)//a.block_size)*a.block_size
    chunks=[stream[i:i+a.block_size] for i in range(0, usable, a.block_size)]
    return {"input_ids":chunks,
            "attention_mask":[[1]*a.block_size for _ in chunks]}
lm=tokenized.map(group, batched=True, batch_size=1000, num_proc=a.num_proc)
lm.save_to_disk("artifacts/tokenized"); print(lm)
```

不要把验证集混进 tokenizer 训练或模型训练。本教程 tokenizer 只看 `train.jsonl`；模型验证只读固定的 `validation` split。

从配置初始化，才是真正从零预训练

Decoder-only Transformer 的数据流



每个位置的 logits 与“右移一位”的真实 token 做交叉熵。因果 mask 保证位置 t 不会偷看未来。注意力核心为 $\mathrm{softmax}((QK^{\top}/\sqrt{d_k})+M)V$，其中未来位置在 M 中被置为负无穷。

模型初始化代码

```
from transformers import AutoTokenizer, GPT2Config, GPT2LMHeadModel
tok = AutoTokenizer.from_pretrained("artifacts/tokenizer")
config = GPT2Config(vocab_size=len(tok), n_positions=256, n_ctx=256,
    n_embd=384, n_layer=6, n_head=6, n_inner=1536,
    activation_function="gelu_new", resid_pdrop=0.1,
    embd_pdrop=0.1, attn_pdrop=0.1,
    bos_token_id=tok.bos_token_id, eos_token_id=tok.eos_token_id,
    pad_token_id=tok.pad_token_id, use_cache=False)
model = GPT2LMHeadModel(config)  # 随机初始化；不是 from_pretrained
n = sum(p.numel() for p in model.parameters())
print(f"parameters={n/1e6:.2f}M")
```

增大参数的代价

- 层数：更深，训练可能更难。
- 隐藏维：参数与计算近似平方增长。
- 序列长：普通 attention 的矩阵近似按 T^2 增长。
- 词表：embedding / LM head 线性增长。

必须相等

- `vocab_size == len(tokenizer)`
- 训练 block ⩽ `n_positions`
- special token IDs 完全一致
- 部署加载同一 tokenizer 目录

训练脚本先固定随机性与协议

`train.py`（模型与参数）

```
import argparse, json, random
from pathlib import Path
import numpy as np, torch
from datasets import load_from_disk
from transformers import (AutoTokenizer, DataCollatorForLanguageModeling,
    GPT2Config, GPT2LMHeadModel, Trainer, TrainingArguments)
def seed_all(seed):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)
ap=argparse.ArgumentParser(); ap.add_argument("--max-steps", type=int, default=5000)
ap.add_argument("--resume", action="store_true"); ap.add_argument("--seed", type=int, default=42)
a=ap.parse_args(); seed_all(a.seed)
tok=AutoTokenizer.from_pretrained("artifacts/tokenizer")
ds=load_from_disk("artifacts/tokenized")
config=GPT2Config(vocab_size=len(tok), n_positions=256, n_ctx=256,
    n_embd=384, n_layer=6, n_head=6, n_inner=1536,
    resid_pdrop=.1, embd_pdrop=.1, attn_pdrop=.1,
    bos_token_id=tok.bos_token_id, eos_token_id=tok.eos_token_id,
    pad_token_id=tok.pad_token_id, use_cache=False)
model=GPT2LMHeadModel(config)
print("params", sum(p.numel() for p in model.parameters()))
use_bf16=torch.cuda.is_available() and torch.cuda.is_bf16_supported()
use_fp16=torch.cuda.is_available() and not use_bf16
```

为什么优先 BF16：它比 FP16 有更大的指数范围，通常更不容易数值溢出；硬件不支持时回退 FP16。PyTorch AMP 会让适合的算子使用低精度，而对敏感算子保留 FP32。

优化设置服务于稳定，而不是堆开关

`train.py`（Trainer、保存与恢复）

```
args=TrainingArguments(output_dir="artifacts/checkpoints", max_steps=a.max_steps,
    per_device_train_batch_size=8, per_device_eval_batch_size=8,
    gradient_accumulation_steps=8, learning_rate=3e-4, weight_decay=.1,
    warmup_ratio=.03, lr_scheduler_type="cosine", max_grad_norm=1.0,
    logging_steps=20, eval_strategy="steps", eval_steps=250,
    save_strategy="steps", save_steps=250, save_total_limit=3,
    load_best_model_at_end=True, metric_for_best_model="eval_loss",
    greater_is_better=False, bf16=use_bf16, fp16=use_fp16,
    gradient_checkpointing=True, report_to=["tensorboard"],
    dataloader_num_workers=4, save_safetensors=True,
    seed=a.seed, data_seed=a.seed)
collator=DataCollatorForLanguageModeling(tokenizer=tok, mlm=False)
trainer=Trainer(model=model, args=args, train_dataset=ds["train"],
    eval_dataset=ds["validation"], data_collator=collator,
    processing_class=tok)
trainer.train(resume_from_checkpoint=True if a.resume else None)
metrics=trainer.evaluate()
out=Path("artifacts/final"); out.mkdir(parents=True, exist_ok=True)
trainer.model.config.use_cache=True
trainer.save_model(out); tok.save_pretrained(out)
(out/"eval_metrics.json").write_text(json.dumps(metrics, indent=2))
```

兼容提示：若旧版 Transformers 不认识 `processing_class`，改为 `tokenizer=tok`；若不认识 `eval_strategy`，升级到锁定版本，而不是在不同机器上静默混用 API。

曲线和断点决定训练是否“可继续”

启动、监控与恢复

```
# 冒烟：先确认 512 steps 能完整保存、评估、生成
python src/train.py --max-steps 512
# 基线：清理冒烟输出或换 output_dir 后运行
python src/train.py --max-steps 5000
# 意外中断：从最近 checkpoint 恢复
python src/train.py --max-steps 5000 --resume
# 单独终端看曲线
tensorboard --logdir artifacts/checkpoints --port 6006
```

每 20-250 steps 看什么

信号	健康表现	动作
train loss	有噪声但总体下降	发散先降 LR；NaN 先查精度/数据
eval loss	下降后趋稳	上升而 train 降：过拟合或泄漏
grad norm	有限且无长期尖峰	持续尖峰：裁剪、降 LR、查异常样本
tokens/s	稳定，无周期性骤降	查 dataloader、磁盘、保存频率
GPU util	数据准备后维持较高	util 低：CPU/tokenization/I/O 喂不满
显存	稳定无阶梯上升	持续增长：缓存或对象泄漏

OOM 处理顺序

1. micro-batch 减半
2. 增加 gradient accumulation
3. 减 block size
4. 开 gradient checkpointing
5. 最后才减模型

checkpoint 应包含

- 模型权重
- optimizer / scheduler state
- global step
- scaler / RNG state
- 训练参数与 tokenizer 引用

评估是多信号验收，不是只看一个 loss

evaluate.py

```
import json, math, torch
from datasets import load_from_disk
from transformers import (AutoModelForCausalLM, AutoTokenizer,
    DataCollatorForLanguageModeling, Trainer, TrainingArguments)
path="artifacts/final"; tok=AutoTokenizer.from_pretrained(path)
model=AutoModelForCausalLM.from_pretrained(path)
ds=load_from_disk("artifacts/tokenized")["validation"]
args=TrainingArguments(output_dir="artifacts/eval", per_device_eval_batch_size=8,
    report_to="none", bf16=torch.cuda.is_available() and torch.cuda.is_bf16_supported())
trainer=Trainer(model=model, args=args, eval_dataset=ds,
    data_collator=DataCollatorForLanguageModeling(tok, mlm=False))
m=trainer.evaluate()
m["perplexity"]=math.exp(m["eval_loss"]) if m["eval_loss"] < 20 else float("inf")
print(json.dumps(m, indent=2))
```

正确解释困惑度

- PPL 是平均 token 负对数似然的指数；越低通常越好。
- 只在相同 tokenizer、相同验证集、相同切块/滑窗策略下横向比较。
- 固定长度模型的 PPL 会受上下文窗口与 stride 影响；报告策略，不只报数字。
- 验证集若与训练近重复，PPL 会虚低。

建议的验收表

维度	最小验收
数值	eval loss 可计算、非 NaN；与上版比较有记录
生成	10-20 个固定提示；无明显无限重复或立即崩坏
复现	固定 seed 的 greedy 输出一致；模型和 tokenizer 哈希可追溯
鲁棒	空输入、超长输入、非 ASCII、恶意大 max_tokens 均被处理
性能	记录 TTFT、tokens/s、并发、峰值显存与 OOM 边界

把生成参数当实验配置保存

generate.py

```
import argparse, torch
from transformers import AutoModelForCausalLM, AutoTokenizer, set_seed
ap=argparse.ArgumentParser(); ap.add_argument("--prompt", default="Once upon a time")
ap.add_argument("--seed", type=int, default=42); a=ap.parse_args(); set_seed(a.seed)
path="artifacts/final"; tok=AutoTokenizer.from_pretrained(path)
dtype=torch.bfloat16 if torch.cuda.is_available() and torch.cuda.is_bf16_supported() else None
model=AutoModelForCausalLM.from_pretrained(path, torch_dtype=dtype)
device="cuda" if torch.cuda.is_available() else "cpu"; model.to(device).eval()
inputs=tok(a.prompt, return_tensors="pt").to(device)
with torch.inference_mode():
    out=model.generate(**inputs, max_new_tokens=120, do_sample=True,
        temperature=.8, top_p=.95, top_k=50, repetition_penalty=1.1,
        pad_token_id=tok.eos_token_id, eos_token_id=tok.eos_token_id)
print(tok.decode(out[0], skip_special_tokens=True))
```

采样参数的作用

参数	作用	常见误区
temperature	缩放 logits；低更保守	不能修复训练不足
top_p	从累计概率质量内采样	与 top_k 同时极严会变僵
top_k	只保留概率最高的 k 个候选	太小导致循环
repetition_penalty	抑制已出现 token	过大损伤语法
max_new_tokens	限制新增 token	不要把总长度当新增长度

固定两套生成：greedy（`do_sample=False`）用于回归；sampling 用于主观质量。每次保存 prompt、seed、参数、模型版本和完整输出。

先做一个边界清楚的最小 API

`serve_api.py`（加载与请求模型）

```
from contextlib import asynccontextmanager
import asyncio, os, torch
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from transformers import AutoModelForCausalLM, AutoTokenizer
MODEL=os.getenv("MODEL_PATH", "artifacts/final")
tok=model=device=None; lock=asyncio.Lock()
class GenIn(BaseModel):
    prompt: str = Field(min_length=1, max_length=4000)
    max_new_tokens: int = Field(default=120, ge=1, le=256)
    temperature: float = Field(default=.8, ge=.05, le=2.0)
    top_p: float = Field(default=.95, gt=0, le=1.0)
@asynccontextmanager
async def lifespan(app: FastAPI):
    global tok, model, device
    tok=AutoTokenizer.from_pretrained(MODEL)
    device="cuda" if torch.cuda.is_available() else "cpu"
    dtype=torch.bfloat16 if device=="cuda" and torch.cuda.is_bf16_supported() else None
    model=AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=dtype)
    model.to(device).eval(); yield
app=FastAPI(title="TinyStory GPT", lifespan=lifespan)
```

为什么加锁：这个最小服务只有一个模型实例。锁让并发请求串行进入 `generate`，避免未经测量的并发把显存推爆。需要动态 batching 与高并发时，应切换 vLLM，而不是盲目增加 Uvicorn workers。

接口能返回，不等于可以裸奔上线

`serve_api.py`（端点）

```
@app.get("/health")
def health(): return {"ok": model is not None, "model": MODEL, "device": device}
@app.post("/generate")
async def generate(x: GenIn):
    if model is None: raise HTTPException(503, "model not ready")
    batch=tok(x.prompt, return_tensors="pt", truncation=True,
              max_length=model.config.n_positions).to(device)
    async with lock:
        with torch.inference_mode():
            ids=model.generate(**batch, do_sample=True,
                max_new_tokens=x.max_new_tokens, temperature=x.temperature,
                top_p=x.top_p, pad_token_id=tok.eos_token_id,
                eos_token_id=tok.eos_token_id)
    return {"text":tok.decode(ids[0], skip_special_tokens=True), "model":MODEL}
```

运行与调用

```
MODEL_PATH=artifacts/final uvicorn src.serve_api:app   --host 127.0.0.1 --port 8000
curl -s http://127.0.0.1:8000/health
curl -s http://127.0.0.1:8000/generate   -H 'content-type: application/json'   -d '{"prompt":"Once upon a
time","max_new_tokens":80}'
```

上线前补齐

- API key / mTLS
- 请求体与 token 限制
- 队列长度与超时
- 结构化日志但不记录敏感正文
- 就绪/存活探针
- 模型版本响应头
- 延迟、吞吐、错误率、OOM 指标
- 反向代理与 TLS

不要直接暴露在公网。示例只绑定 `127.0.0.1`；若绑定 `0.0.0.0`，必须在前面加认证、限流、TLS 与网络访问控制。

需要并发时，把服务层交给 vLLM

> vLLM 提供 OpenAI 兼容 HTTP 服务与连续批处理，适合 NVIDIA GPU 上的多请求推理。官方文档建议用新环境安装，以减少 CUDA/PyTorch 二进制冲突。

独立环境与启动

```
python3 -m venv .venv-vllm
source .venv-vllm/bin/activate
python -m pip install -U pip
pip install vllm
vllm serve ./artifacts/final   --served-model-name tinystory-gpt --dtype auto   --max-model-len 256 --generation-config vllm   --host 127.0.0.1 --port 8000
```

基础模型调用 Completions API

```
curl http://127.0.0.1:8000/v1/completions   -H 'Content-Type: application/json'   -d '{"model":"tinystory-gpt","prompt":"Once upon a time",       "max_tokens":120,"temperature":0.8,"top_p":0.95}'
```

为什么不是 Chat API

本模型没有指令微调，也没有 tokenizer chat template。vLLM 的 Chat Completions 需要聊天模板来编码 role/message；硬塞模板不会自动获得聊天能力。

什么时候用 tensor parallel

单卡放不下模型或需要多卡分摊时再加 `--tensor-parallel-size N`。先确认多卡互联与 NCCL 正常，并实测吞吐是否真的提升。

压测至少记录

并发数、输入/输出 token 分布、TTFT、TPOT、请求吞吐、token 吞吐、P50/P95/P99、峰值显存、超时与 OOM 比例。

容器固化运行时，版本号保障回滚

可移植 CPU 容器（最小示例）

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
RUN pip install --no-cache-dir torch transformers safetensors fastapi "uvicorn[standard]"
COPY artifacts/final /model
COPY src/serve_api.py /app/serve_api.py
ENV MODEL_PATH=/model
USER 65532:65532
EXPOSE 8000
CMD ["uvicorn","serve_api:app","--host","0.0.0.0","--port","8000"]
```

```
docker build -t tinystory-gpt:0.1 .
docker run --rm -p 127.0.0.1:8000:8000   --read-only --tmpfs /tmp --memory 4g --cpus 2 tinystory-gpt:0.1
```

GPU 容器不要照抄任意 CUDA 基础镜像。选择与你的驱动、PyTorch 或 vLLM 版本匹配的官方镜像，并把 tag 或 digest 固定；在同一目标 GPU 上做冷启动、并发和重启测试。

一次可回滚发布

1. **构建：**模型目录只含 config、tokenizer、generation config、safetensors 与 model card。
2. **验收：**镜像内运行固定提示回归与健康检查。
3. **灰度：**新旧版本同时在线，小流量验证错误率与延迟。
4. **切换：**用不可变版本号更新路由，不覆盖旧目录。
5. **回滚：**指标越界立即把路由指回上一版本；不要现场修容器。

推荐版本标签

`model: tinystory-gpt-14m` · `data: ts-2026-10-s200k` · `tokenizer: bpe8k-v1` · `image: serving-0.1.0`。

扩规模是基于证据选择并行策略

先测瓶颈，再选并行方式

现象	首选动作	不要先做
GPU util 低、CPU 满	预分词、增 workers、顺序读、NVMe cache	加 GPU
单卡 OOM，计算仍足	减 micro-batch、检查点、FSDP/ZeRO	随意关混合精度
模型能放下，训练太慢	DDP 数据并行；提高每步 token 数	先上 tensor parallel
单卡放不下模型	FSDP/ZeRO；再看 tensor/pipeline parallel	保留完整 optimizer 副本
数据超磁盘	streaming + buffer shuffle + shard	每 epoch 全量重下载

多卡起点

```
accelerate config
accelerate launch src/train.py --max-steps 5000
# 更大模型需要 FSDP 时，使用明确的 accelerate 配置文件；
# 确保保存 FULL_STATE_DICT 供 from_pretrained/vLLM 加载。
```

DDP

每卡完整模型，各自处理不同 batch，再同步梯度。模型能放入单卡时最简单，通常也是第一选择。

FSDP

参数、梯度、optimizer states 在多卡分片，可显著降低每卡模型状态占用，但保存、恢复和调试更复杂。

扩展实验矩阵

实验	只改这一项	观察
A	200k → 500k 样本	eval loss / 固定生成是否继续改善
B	14M → 90M 参数	相同 token 预算下是否欠训练
C	block 256 → 512	一致性与 tokens/s/显存
D	BPE 8k → 16k	序列长度与 embedding 成本

排错从最小闭环开始

症状 → 检查 → 修复

症状	先检查	优先修复
CUDA 不可用	`nvidia-smi`、torch、CUDA available	重建 venv；安装匹配 wheel
一开始就 OOM	batch、block、模型维度、残留进程	batch 减半；清理进程；开 checkpoint
loss=NaN	异常文本、LR、FP16 溢出、grad norm	切 BF16/FP32；降 LR；保留失败 batch
loss 不降	labels、词表/ID、参数是否更新	对 100 条过拟合；打印梯度
生成乱码	tokenizer 是否同一个；decode	比对哈希和 special IDs
无限重复	训练不足、采样、EOS 学习	看 greedy；加数据/steps；检查 EOS
resume 曲线跳变	optimizer/scheduler/RNG 是否恢复	用 Trainer checkpoint，不只加载权重
验证异常好	train/valid 重复、同源切分	先去重后切分；外部测试集
服务延迟抖动	长度、排队、冷启动	限 token；预热；切 vLLM batching
vLLM 启动失败	架构、CUDA/PyTorch ABI、目录完整	新环境；查兼容矩阵；回退 Transformers

100 条样本过拟合测试

在正式跑前，用极小数据关闭 dropout、训练到 loss 明显下降。若做不到，问题大概率在数据/label/模型连接，而不是“数据太少”。这个测试比盲调学习率更快定位基本错误。

> **排错原则：**从输入 JSONL → token IDs → batch shape → 单步 forward/backward → checkpoint → generate → HTTP，逐层验证；不要同时修改数据、模型和优化器。

生产化是模型之外的一整套门禁

上线前门禁

模型与数据

- ✓ 数据来源、许可、过滤、切分可审计
- ✓ 模型卡写明用途、限制、已知偏差
- ✓ 固定提示回归已通过
- ✓ tokenizer、config、权重哈希已记录
- ✓ 有明确旧版本和回滚动作

服务与安全

- ✓ 认证、TLS、限流、输入/输出 token 上限
- ✓ 超时、队列上限、并发与 OOM 降级
- ✓ 日志默认不存 prompt 正文
- ✓ 网络只开放必要端口
- ✓ 依赖镜像与模型文件做完整性检查

可观测性

- ✓ 健康与就绪探针分开
- ✓ 请求数、错误率、TTFT、TPOT、tokens/s
- ✓ P50/P95/P99 与超时率
- ✓ GPU util、显存、温度、功耗
- ✓ 按模型版本聚合指标

业务边界

- ✓ 用户知道生成模型可能出错
- ✓ 高风险用途有人类复核
- ✓ 滥用检测与封禁流程存在
- ✓ 数据保留期与删除流程明确
- ✓ 上线地域符合法规和许可

建议的 SLO 起草法

先用真实输入长度和并发压测，再定 SLO。至少写清：可用性、P95 首 token 延迟、P95 完成延迟、最大输入/输出 tokens、每用户速率、OOM/超时预算、回滚阈值。

> **模型输出不可信任。**对下游执行代码、SQL、命令、支付、医疗或安全决策，必须增加结构校验、最小权限、沙箱和人工确认。

从实验模型走向可用系统

四个里程碑

M1 / 当天

20k 样本、512 steps；能保存、加载、续训、生成。

M2 / 基线

200k 样本、固定验证集；记录 loss/PPL/提示集与资源曲线。

M3 / 服务

FastAPI 回归通过；vLLM 压测；认证、限流、监控和回滚就绪。

M4 / 扩展

自有许可语料；90M 或更大；DDP/FSDP；独立测试集与模型卡。

接下来学什么

目标	下一步	注意
变成助手	基础模型上做 SFT，加入 chat template	高质量指令数据比模板重要
回答私有知识	先做 RAG，再评估继续预训练	知识可更新、可引用、可撤回
适应领域语言	继续预训练（DAPT）后再 SFT	混通用数据，监测遗忘
降低显存	量化用于推理；LoRA 用于微调	不是从零预训练替代品
中文模型	合法中文语料，重训 tokenizer	比较 16k/32k 与序列长度

一次实验复盘模板

```
实验 ID：model14m-data200k-bpe8k-seed42
假设：增加训练 steps 会继续降低 eval loss
唯一改动：max_steps 5000 → 10000
固定项：数据快照、tokenizer、model config、batch tokens
结果：best eval loss / PPL / prompts / tokens/s / peak VRAM
决定：保留、回滚，或下一次只改一个变量
```

成功标准：任何人拿到相同快照、配置与命令，都能解释训练发生了什么、重现结果并安全地调用服务。

把全流程压缩成可执行命令

requirements.txt

```
# torch 单独按目标 CPU/CUDA 环境安装
transformers>=4.49,<6
datasets>=3,<5
tokenizers>=0.20
accelerate>=1
safetensors>=0.4
tensorboard>=2.16
fastapi>=0.115
uvicorn[standard]>=0.30
```

一页命令表

```
python3 -m venv .venv && source .venv/bin/activate
pip install -U pip wheel
# 按官方选择器安装 torch；再 pip install -r requirements.txt
python src/prepare_data.py --train 200000 --valid 5000
python src/train_tokenizer.py --vocab-size 8000
python src/tokenize_data.py --block-size 256 --num-proc 4
python src/train.py --max-steps 512          # 冒烟
python src/train.py --max-steps 5000         # 基线
python src/train.py --max-steps 5000 --resume
python src/evaluate.py
python src/generate.py --prompt "Once upon a time"
MODEL_PATH=artifacts/final uvicorn src.serve_api:app   --host 127.0.0.1 --port 8000
vllm serve ./artifacts/final --served-model-name tinystory-gpt   --max-model-len 256 --generation-config vllm --host
127.0.0.1
```

第一次不要跳过冒烟。512 steps 的目的不是质量，而是验证完整生命周期：训练 → checkpoint → resume → final export → generation → API。

术语与边界

术语速查

Token
模型处理的离散 ID，不等于字符或单词。

Context length
一次前向可使用的最大 token 数。

Logits
softmax 前对词表每个 token 的未归一化分数。

Cross-entropy
预测分布与真实下一 token 的损失。

Perplexity
平均 token loss 的指数，只在同协议下可比。

Effective batch
micro-batch × 累积步数 × GPU 数。

AMP
自动混合精度；按算子选择低精度或 FP32。

Checkpoint
用于恢复训练的模型、优化器、调度器与状态。

TTFT
Time To First Token，首 token 延迟。

TPOT
Time Per Output Token，输出阶段单位 token 时间。

DDP
数据并行；每卡一份模型，同步梯度。

FSDP
完全分片数据并行；分片参数、梯度和优化器状态。

三条容易混淆的边界

1. **预训练 ≠ 微调：**本教程随机初始化全部权重；LoRA/QLoRA 通常从已有模型继续。
2. **模型格式 ≠ 推理引擎：**`save_pretrained` 产物可被 Transformers 或支持该架构的 vLLM 读取。
3. **API 兼容 ≠ 行为兼容：**OpenAI 兼容端点只代表请求/响应形状近似，不代表模型具有同等能力。

参考资料与版本说明

核心资料

1. Vaswani et al. *Attention Is All You Need*. arxiv.org/abs/1706.03762v1
2. Eldan & Li. *TinyStories*. arxiv.org/pdf/2305.07759
3. TinyStories 数据集卡。huggingface.co/datasets/roneneldan/TinyStories
4. HF：Training a causal language model from scratch. huggingface.co/learn/llm-course/chapter7/6
5. HF Tokenizers：Training from memory. huggingface.co/docs/tokenizers/en/training_from_memory
6. HF：Causal language modeling. huggingface.co/docs/transformers/tasks/language_modeling
7. HF：Perplexity. huggingface.co/docs/transformers/en/perplexity
8. PyTorch：Automatic Mixed Precision. docs.pytorch.org/docs/main/amp.html
9. PyTorch：Saving and Loading Models. docs.pytorch.org/.../saving_loading_models.html
10. vLLM：OpenAI-Compatible Server. docs.vllm.ai/.../openai_compatible_server/
11. vLLM：GPU Installation. docs.vllm.ai/.../installation/gpu/
12. HF Accelerate Quicktour. huggingface.co/docs/accelerate/main/quicktour
13. HF Transformers：FSDP. huggingface.co/docs/transformers/en/fsdp

> **版本说明：**开源库 API 和硬件支持会变化。第一次跑通后应保存 `requirements.lock.txt`、镜像 digest、模型/数据/tokenizer 哈希。

使用建议

先按 14M/200k 路线原样跑通，再替换自己的合法语料。若目标是业务问答，通常先做 RAG 或在成熟开源基座上做 SFT；本教程的价值是把预训练和部署的底层闭环真正做一遍。